Database and MySQL Handbook

Trinethr N

Table of contents

Preface **1**
Disclaimer . . . 1
AI Usage Disclosure . . . 1
Feedback . . . 1

1 Introduction to Databases **2**
1.1 Advantages . . . 2
1.2 Database Models . . . 2
1.3 The Relational Database Model . . . 2
1.3.1 Structure of a Relation . . . 3
1.3.2 Degree and Cardinality . . . 3
1.3.3 Keys . . . 3
1.4 Relational Database Management Systems (RDBMS) . . . 4
1.4.1 Schema . . . 4
1.4.2 MySQL . . . 4

2 Structured Query Language (SQL) **5**
2.1 Categories of SQL Commands . . . 5
2.2 Structure of a command . . . 5

3 Creating and Using a Database **7**
3.1 Other Important Commands . . . 7

4 Datatypes in MySQL **8**
4.1 Numeric Types . . . 8
4.1.1 Notes . . . 8
4.2 Date/Time Types . . . 8
4.2.1 Notes . . . 9
4.3 Text Types . . . 9
4.3.1 Notes . . . 9
4.4 Other Types . . . 10
4.4.1 Notes . . . 10

5 Working with Tables **11**
5.1 Creating Tables . . . 11
5.2 Describing Tables . . . 12
5.3 Constraints . . . 12
5.3.1 Column-Level Constraints . . . 13
5.3.2 Table-Level Constraints . . . 13
5.3.3 Commonly used onstraints . . . 13
5.3.4 Foreign Keys . . . 14
5.3.5 Naming Constraints (Optional) . . . 15

5.4 Altering Tables . . . 15
5.4.1 Adding and Removing Columns . . . 16
5.4.2 Adding/Removing Constraints . . . 17
5.4.3 Renaming and Modifying Columns . . . 18
5.5 Deleting Tables . . . 19
5.5.1 Important Note . . . 19

6 Data Manipulation 20
6.1 Inserting Data . . . 20
6.1.1 Inserting Values for All Columns . . . 20
6.1.2 Inserting Values for Selected Columns . . . 20
6.2 Updating Data . . . 21
6.2.1 Examples . . . 21
6.2.2 Important Note . . . 22
6.3 Deleting Data . . . 22
6.3.1 Examples . . . 23
6.3.2 Important Note . . . 23

7 Introduction to Queries 24
7.1 Arithmetic and Logical Operations . . . 24
7.2 Working with Strings . . . 26
7.3 MySQL's Built-In Functions . . . 26
7.3.1 Numeric Functions . . . 26
7.3.2 Date/Time Functions . . . 26
7.3.3 String Functions . . . 27
7.3.4 Miscellaneous . . . 27
7.3.5 Using Built-In functions without a table . . . 28
7.3.6 Importance . . . 28

8 Querying Individual Tables 29
8.1 Selecting Unique Values . . . 29
8.2 Specifying Columns . . . 29
8.2.1 Aliases . . . 30
8.3 The FROM Clause . . . 30
8.4 The WHERE Clause . . . 30
8.4.1 The LIKE operator . . . 31
8.5 Aggregate Functions and the GROUP BY Clause . . . 32
8.5.1 Notes . . . 33
8.5.2 Example . . . 33
8.6 The HAVING Clause . . . 34
8.6.1 Example . . . 34
8.6.2 Common Mistakes . . . 34
8.6.3 Using both WHERE and HAVING . . . 34
8.7 The ORDER BY Clause . . . 34
8.7.1 Examples . . . 35
8.8 The LIMIT Clause . . . 35
8.8.1 Examples . . . 35

8.9 Order of Execution of the SELECT command . . . 36

9 Querying Multiple Tables — Joins 37
9.1 Introduction to Joins . . . 37
9.2 Cross Join (Cartesian Product) . . . 37
9.2.1 Syntax . . . 37
9.2.2 Example . . . 37
9.3 Equi-Join — Joining based on an equality condition . . . 38
9.3.1 Classification . . . 38
9.3.2 Syntax . . . 39
9.3.3 Example . . . 39
9.3.4 Table Aliases . . . 40
9.4 Natural Join . . . 41
9.4.1 Syntax . . . 41
9.4.2 Example . . . 41
9.5 Note . . . 41

10 Afterword 42
10.1 Some More Information . . . 42
10.2 Errors in the Handbook . . . 42

Preface

This book was written with an aim to explain the concepts of Database Management with MySQL, according to the CBSE Syllabus, in a concise manner with good flow of topics.

It introduces the concept of databases, explaining its advantages over traditional data storage systems, and explains the relational database model. Following this is a comprehensive guide to working with MySQL - including creating databases and tables, adding data, and fetching data with queries.

Disclaimer

This document has been written in accordance to the 2025-26 CBSE Senior Secondary Computer Science Syllabus, to cover topics in *Unit 3: Database Management* (**except 'interfacing an SQL database with Python'**), however it should be used as a supplementary resource alongside the prescribed textbook and other study materials.

AI Usage Disclosure

AI tools have been used to edit some parts of the book, such as some examples and descriptions, to enhance their clarity and structure.

Feedback

You can write your feedback regarding the handbook to me at trinethr_n@yahoo.com. Alternatively, if you have corrections or suggestions that you would like to implement, fork the github repository at brinterwastaken/db-mysql-handbook and open a pull request.

1 Introduction to Databases

A database is a system of organized storage of data that allows easy storage, updating, and retrieval of data according to a specific use case. Databases are essential for *storing, managing, and retrieving massive volumes of structured data efficiently*, which is impossible with traditional file systems or spreadsheets. They provide data integrity, security, and fast access, enabling applications like social media, banking systems, and online shopping to function reliably.

1.1 Advantages

- **Efficient Data Management & Storage** - Structured storage of data (e.g. in rows and columns), allowing for easier management.
- **Fast Information Retrieval** - Many databases use indexing allowing for fast retrieval of specific data from large data sets.
- **Data Integrity & Consistency** - Databases enforce rules and constraints to prevent data duplication and errors, ensuring that data is accurate and consistent across all applications.
- **Data Security and Access** - Data can be secured to allow access only for specific users. Simultaneous access of data can also be implemented using databases.
- **Backup & Recovery** - Databases allow easy setup of backup tools to prevent accidental data losses.

1.2 Database Models

- A database model is a theoretical framework determining the logical structure of a database, governing how data is stored, organized, and manipulated.
- There are many different database models that are suited for different applications. One of the most widely used is the **relational database model**, which is both simple and versatile.

1.3 The Relational Database Model

- This model organizes data into **tables**, consisting of **rows** (also called *tuples*) and **columns** (also called *attributes*).
- Each column contains values of the same type of information, while each row contains values of different attributes describing a single item.

For example:

Id	Brand	Model	Color
001	Maruti Suzuki	Dzire	White
002	Tata	Sierra	Yellow
003	Hyundai	Creta	Grey
004	Toyota	Hyrider	Black

Id	Brand	Model	Color
005	Mahindra	XUV700	Dark Blue

In the above example, each row represents one vehicle, while each column contains a single type of information about the vehicles.

1.3.1 Structure of a Relation

- For any attribute, we can define a **domain** as the set of all possible values that attribute may take.
- Let the domain of attribute A_n be D_n.
- A relation consists of tuples with *n* values corresponding to the *n* attributes of the relation. Thus, each tuple belongs to

$$D_1 \times D_2 \times D_3 \times ... \times D_n$$

Since not every possible combination necessarily appears in the table, the relation itself is a **subset** of this Cartesian product.

1.3.2 Degree and Cardinality

Degree - The **number of attributes (columns)** in a relation is called its **degree**. - In the above table the degree is **4**.

Cardinality - The **number of tuples (rows)** in a relation is called its **cardinality**. - In the above table the cardinality is **5**.

1.3.3 Keys

Keys are an essential part of the relational model, as they allow rows in a table to be uniquely identified. A key may consist of a single column or a combination of columns whose values satisfy specific constraints.

1. **Primary Key**
   A column or set of columns that uniquely identifies each record in a table.
   Primary keys **cannot contain NULL values**.

2. **Unique Key**
   A column or set of columns that ensures all values are unique.
   Unlike a primary key, a unique key **may allow a NULL value** depending on the database system.

3. **Foreign Key**
   A column (or set of columns) that establishes a relationship between two tables by referencing the **primary key of another table**.

4. **Candidate and Alternate Keys**

   - *Candidate keys* are columns or sets of columns that can uniquely identify each record in a table.

- One candidate key is chosen by the developer as the *primary key*.
- The remaining candidate keys are called *alternate keys*.

1.4 Relational Database Management Systems (RDBMS)

- The relational model describes the *theoretical structure* of relational databases. In practice, databases based on this model are implemented using **Relational Database Management Systems (RDBMS)**.
- An RDBMS is software that allows users to **create, manage, and query relational databases** while enforcing rules such as data integrity, access control, and relationships between tables.
- Some widely used RDBMS implementations include **MySQL, PostgreSQL, SQLite, Oracle Database, and Microsoft SQL Server**.

1.4.1 Schema

- In a database management system (DBMS), a **schema** is the skeleton structure or "blueprint" that defines how data is organized. It outlines the logical constraints, relationships, and objects that make up the database, such as tables, views, fields, and indexes.

1.4.2 MySQL

- **MySQL** is a widely used **open-source relational database management system**. It is known for its reliability, performance, and ease of use, and is commonly used in web applications.
- In MySQL, data is organized into **databases containing tables**, following the relational model described earlier. Users interact with these databases using **Structured Query Language (SQL)**, which allows them to create tables, insert data, and retrieve information. In the following sections, we will introduce **SQL and its basic commands**.

2 Structured Query Language (SQL)

- To interact with relational databases, we use **Structured Query Language (SQL)**. SQL is a standard language used to *create, modify, and query databases*.

2.1 Categories of SQL Commands

- SQL commands are generally categorized into several groups:
  1. **Data Definition Language (DDL)** - Commands used to *define or modify* the **structure of the database**. Example commands include `CREATE`, `ALTER` and `DROP`. Other commands such as `GRANT`, `REVOKE` modify the permissions users have to access tables, and are grouped as a subcategory called Data Control Language (DCL).
  2. **Data Manipulation Language (DML)** - Commands used to *insert, modify, and delete data* stored in tables. Example commands include `INSERT`, `UPDATE` and `DELETE`.
  3. **Data Query Language (DQL)** - The `SELECT` command is used to query data from tables of the database.
  4. **Transaction Control Language (TCL)** - Commands used to manage **database transactions**, ensuring data integrity during multiple operations. Example commands include `COMMIT`, `ROLLBACK`, and `SAVEPOINT`.
- SQL keywords are case insensitive, but developers often write these keywords in `UPPERCASE`, to distinguish them from identifiers such as table names or column names. This document will follow the uppercase convention.

2.2 Structure of a command

An SQL command generally consists of **keywords, identifiers, and values** arranged in a specific order to perform an operation on the database.

- **Keywords** are *reserved words* that define the operation to be performed, such as `SELECT`, `INSERT`, `UPDATE`, `DELETE`, and `FROM`.
- **Identifiers** are the names of database objects such as *tables, columns, or databases*.
- **Values/Literals** are the actual data being inserted, updated, or compared in a query.
- **Clauses** are sections of a command that perform specific roles, such as `WHERE`, `ORDER BY`, or `GROUP BY`.

Example:

```
SELECT Brand, Model FROM Vehicles WHERE Color = 'Black';

/* This can be broken down into clauses as: */
SELECT Brand, Model
│      │      │
```

```
│       │       └── column name (identifier)
│       └─────────── column name (identifier)
└─────────────────── keyword
FROM Vehicles
│    │
│    └──────────── table name (identifier)
└─────────────────── keyword
WHERE Color = 'Black';
│     │       │
│     │       └── value
│     └─────────── column name (identifier)
└─────────────────── keyword (clause)
```

- Each SQL command must be terminated by a semicolon `;`.
- Comments (statements ignored by the RDBMS) can be entered in commands by using `-- Comment` or `/* Comment */`. The latter can be used for multi line comments.

These commands should be supplied to the RDBMS's command line client. MySQL's command line client has a prompt `mysql>` which indicates where the command is to be entered. Thus the above command when entered in the command line client looks like:

```
mysql> SELECT Brand, Model FROM Vehicles WHERE Color = 'Black';
```

In this document, when we mention a command without its output, we only show the command itself, but when the output is also given, the `mysql>` prompt is also shown to highlight the command executed.

3 Creating and Using a Database

The `CREATE DATABASE` command is used to create a new database.

Syntax:

```
CREATE DATABASE name;
```

Here, `name` is a **required** parameter, which gives the newly created database a name.

- Spaces can be put in the database name, but special syntax must be used for this, and this is generally discouraged.

To use the newly created database, we use the `USE` command:

```
USE name;
```

This command sets the current active database to the database specified by `name`.

Most commands outlined in this document are to be executed with an active database, i.e. we must `USE` a database before executing them.

3.1 Other Important Commands

These are commands often used while creating/using databases:

1. `SHOW DATABASES` - Shows a list of databases that are present in a MySQL installation.
2. `SHOW TABLES` - Shows a list of tables within a database. (Requires to be in an active database with `USE` as shown above)

4 Datatypes in MySQL

- To organize data, we must create tables according to the data that we require them to store.
- Since tables should have multiple columns storing different types of data, MySQL has some built in datatypes, from which the appropriate datatype for a column can be chosen.

4.1 Numeric Types

Data Type	Storage	Range (Signed)	Range (Unsigned)	Description
`TINYINT`	1 byte (8 bits)	-2^7 to $2^7 - 1$	0 to $2^8 - 1$	Very small integers
`SMALLINT`	2 bytes (16 bits)	-2^{15} to $2^{15} - 1$	0 to $2^{16} - 1$	Small integers
`MEDIUMINT`	3 bytes (24 bits)	-2^{23} to $2^{23} - 1$	0 to $2^{24} - 1$	Medium-sized integers
`INT` / `INTEGER`	4 bytes (32 bits)	-2^{31} to $2^{31} - 1$	0 to $2^{32} - 1$	Standard integer type
`BIGINT`	8 bytes (64 bits)	-2^{63} to $2^{63} - 1$	0 to $2^{64} - 1$	Large integers
`FLOAT`	4	Approx. $\pm 3.4 \times 10^{38}$	Same	Single-precision floating-point numbers
`DOUBLE`	8	Approx. $\pm 1.7 \times 10^{308}$	Same	Double-precision floating-point numbers
`DECIMAL(M,D)`	Variable	Depends on precision	Depends on precision	Exact fixed-point numbers (often used for financial data)
`BIT(M)`	1–8	Depends on M	Depends on M	Bit-value type for storing binary values

4.1.1 Notes

- `M`: Precision - total number of digits
- `D`: Scale - number of digits after the decimal point Default `(M,D) = (10,0)` for `DECIMAL` type.
- Numeric datatypes are signed by default (can hold negative values), but they can be specified to be unsigned (if negative values are not required to be stored in the column), which also doubles the range as shown above.

4.2 Date/Time Types

Data Type	Storage (Bytes)	Format	Range	Description
`DATE`	3	`YYYY-MM-DD`	1000-01-01 to 9999-12-31	Stores a calendar date without time
`TIME`	3	`HH:MM:SS`	-838:59:59 to 838:59:59	Stores a time of day or time interval
`DATETIME`	8	`YYYY-MM-DD HH:MM:SS`	1000-01-01 00:00:00 to 9999-12-31 23:59:59	Stores both date and time
`TIMESTAMP`	4	`YYYY-MM-DD HH:MM:SS`	1970-01-01 00:00:01 UTC to 2038-01-19 03:14:07 UTC	Stores date and time relative to Unix epoch
`YEAR`	1	`YYYY`	1901 to 2155 (and 0000)	Stores a year value

4.2.1 Notes

- **`DATETIME` vs `TIMESTAMP`**:
  - `DATETIME` stores the value **exactly as given**.
  - `TIMESTAMP` is stored internally as **Unix time** and is **automatically converted based on time zones**.
- `TIMESTAMP` is commonly used for columns storing record creation/updation times.

4.3 Text Types

Data Type	Storage	Range	Description
`CHAR(n)`	Fixed-length	0–255 characters	Stores fixed-length strings; unused space is padded
`VARCHAR(n)`	Variable-length	Up to 65,535 bytes (row limit applies)	Stores variable-length strings
`TINYTEXT`	Variable	255 bytes	Very small text strings
`TEXT`	Variable	65,535 bytes (~64 KB)	Standard text storage
`MEDIUMTEXT`	Variable	16,777,215 bytes (~16 MB)	Medium-sized text data
`LONGTEXT`	Variable	4,294,967,295 bytes (~4 GB)	Very large text data

4.3.1 Notes

- **`CHAR(n)`**
  - Uses **fixed storage size**.
  - If the stored string is shorter than n, MySQL **pads it with spaces**.
  - By default, n is 1 (if written as `CHAR`).
- **`VARCHAR(n)`**
  - Uses **only as much storage as needed**, making it more efficient for most text fields.
  - The n parameter must be specified for the `VARCHAR` type.

- **`TEXT` types**
  - Used for storing **large blocks of text**, such as articles, descriptions, or logs.
  - Unlike `CHAR` and `VARCHAR`, they usually **cannot have default values** in many MySQL versions.

4.4 Other Types

Data Type	Storage	Description
`ENUM`	1–2 bytes	A string object that can take **one value from a predefined list**
`BINARY(n)`	Fixed-length	Stores fixed-length **binary data**
`TINYBLOB`	Up to 255 bytes	Used for storing **very small binary data**, such as small files or binary identifiers
`BLOB`	Up to 65,535 bytes (~64 KB)	Used for storing **binary data** such as *images, files, or encoded data*
`MEDIUMBLOB`	Up to 16,777,215 bytes (~16 MB)	Used for storing **larger binary objects**, such as *medium-sized media files*
`LONGBLOB`	Up to 4,294,967,295 bytes (~4 GB)	Used for storing **very large binary data**, such as *large files or multimedia content*

4.4.1 Notes

- Enum is useful for **fields with a small fixed set of options**, e.g. a status column having possible values — `pending`, `approved` or `rejected`.

5 Working with Tables

- We can create tables using the `CREATE TABLE` command, specifying the columns according to our requirement.
- The created tables can also be altered later to edit columns, change constraints, etc.
- These operations will be focused on in this chapter.

5.1 Creating Tables

- The `CREATE TABLE` command is used to create a new table.

Syntax:

```
CREATE TABLE tablename (
    column1,        ┐
    [column2,]      ├─ Table columns
    [...]           ┘
    [constraint1],  ┐
    [constraint2]   ├─ Optional constraints for columns
    [...]           ┘
);
```

- In the above syntax, `column1`, `column2`, etc. refer to column definitions. The syntax for these is outlined below.

Column definition Syntax:

```
column_name datatype [key type] [constraints]
│           │        │         │
│           │        │         └── Additional rules applied to the column
│           │        │             (e.g., NOT NULL, DEFAULT, CHECK)
│           │        │
│           │        └──────────── Key specification
│           │                      (e.g., PRIMARY KEY, UNIQUE, FOREIGN KEY)
│           │
│           └─────────────────── Data type of the column
│                                (e.g., INT, VARCHAR(50), DATE)
│
└──────────────────────────────── Name of the column (identifier)
```

Example:

```
CREATE TABLE Customers (
    CustomerId VARCHAR(20) PRIMARY KEY,
    CustomerName VARCHAR(50) NOT NULL,
    Address VARCHAR(150) NOT NULL,
    PhoneNumber VARCHAR(10),
    Email VARCHAR(50) NOT NULL,
    DateOfBirth DATE
);
```

5.2 Describing Tables

- The DESC command can be used to describe the structure of an existing table
- It shows a list of columns along with their datatypes, default values, type of key, and other information about the column.

Syntax:

```
DESC tablename;
```

Example:

```
mysql> DESC Customers;
+--------------+--------------+------+-----+---------+-------+
| Field        | Type         | Null | Key | Default | Extra |
+--------------+--------------+------+-----+---------+-------+
| CustomerId   | varchar(20)  | NO   | PRI | NULL    |       |
| CustomerName | varchar(50)  | NO   |     | NULL    |       |
| Address      | varchar(150) | NO   |     | NULL    |       |
| PhoneNumber  | varchar(10)  | YES  |     | NULL    |       |
| Email        | varchar(50)  | NO   |     | NULL    |       |
| DateOfBirth  | date         | YES  |     | NULL    |       |
+--------------+--------------+------+-----+---------+-------+
```

5.3 Constraints

In a lot of cases, we require that all values of a column follow a particular set of rules. These can be enforced using constraints, which prevent insertion of data that do not follow the defined rules.

Constraints can be defined in **two ways** when creating a table:

1. **Column-level constraints** – written directly in the column definition.
2. **Table-level constraints** – written after all column definitions.

5.3.1 Column-Level Constraints

These are specified **immediately after the datatype of the column**. They apply only to that particular column.

Example:

```
CREATE TABLE Products (
    ProductId INT PRIMARY KEY,
    ProductName VARCHAR(100) NOT NULL,
    Price DECIMAL(10,2) CHECK (Price > 0)
);
```

Here:

- `PRIMARY KEY` is applied to `ProductId`
- `NOT NULL` is applied to `ProductName`
- `CHECK` ensures that `Price` must be greater than 0

5.3.2 Table-Level Constraints

Constraints can also be defined **after the list of columns**, inside the table definition. This is useful when:

- A constraint involves **multiple columns**
- You want to **separate structure from constraints** for clarity

Example:

```
CREATE TABLE Products (
    ProductId INT,
    ProductName VARCHAR(100),
    Price DECIMAL(10,2),

    PRIMARY KEY (ProductId)
);
```

Here the `PRIMARY KEY` constraint is defined **after the column definitions**, rather than inside the column definition.

5.3.3 Commonly used onstraints

Constraint	Meaning	Usage
`NOT NULL`	Ensures the column cannot have a null value	`age INT NOT NULL`

Constraint	Meaning	Usage
`UNIQUE`	Ensures all values in the column are distinct	`email VARCHAR(100) UNIQUE`
`PRIMARY KEY`	Uniquely identifies each row; implies NOT NULL and UNIQUE	`id INT PRIMARY KEY`
`DEFAULT`	Sets a default value if none is provided during insert	`city VARCHAR(50) DEFAULT 'Unknown'`
`CHECK`	Ensures values in the column satisfy a given condition	`age INT CHECK (age >= 18)`
`FOREIGN KEY`	Links a column to the primary key of another table, more information below	`FOREIGN KEY (student_id) REFERENCES students(id)`

5.3.4 Foreign Keys

- A **foreign key** is used to create a relationship between two tables. It ensures that values in one table **correspond to existing values in another table**.
- A foreign key references the **primary key of another table**.

5.3.4.1 Example Tables

Suppose we have a table of customers:

```
CREATE TABLE Customers (
    CustomerId VARCHAR(20) PRIMARY KEY,
    CustomerName VARCHAR(50)
);
```

Now we create another table that refers to customers. The foreign key can be specified in one of two ways, similar to other constraints:

A) Inline with column definition - using the `REFERENCES` keyword

```
CREATE TABLE Orders (
    OrderId INT PRIMARY KEY,
    CustomerId VARCHAR(20) REFERENCES Customers(CustomerId),
    OrderDate DATE
);
```

B) Separated from column definition - using the `FOREIGN KEY` clause

```
CREATE TABLE Orders (
    OrderId INT PRIMARY KEY,
    CustomerId VARCHAR(20),
    OrderDate DATE,
```

```
    FOREIGN KEY (CustomerId) REFERENCES Customers(CustomerId)
);
```

This ensures that every `CustomerId` in the `Orders` table **must exist in the `Customers` table**.

5.3.5 Naming Constraints (Optional)

- Constraints can also be given **explicit names**, which helps when modifying or removing them later (this will be discussed in further sections).
- Constraint names are typically specified when constraints are written separately from the column definitions using the `CONSTRAINT` keyword.

Example:

```
CREATE TABLE Orders (
    OrderId INT,
    CustomerId VARCHAR(20),

    CONSTRAINT pk_orders PRIMARY KEY (OrderId),
    CONSTRAINT fk_customer
        FOREIGN KEY (CustomerId)
        REFERENCES Customers(CustomerId)
);
```

Here:

- `pk_orders` is the name of the primary key constraint
- `fk_customer` is the name of the foreign key constraint

5.4 Altering Tables

- The `ALTER TABLE` command is used to **modify the structure of an existing table**.
- It allows operations such as:
  - adding new columns
  - removing existing columns
  - modifying column definitions
  - adding or removing constraints

Syntax:

```
ALTER TABLE tablename
    action1,  ┐
    [action2] ├─ One or more modification operations
    [ ... ]   ┘
;
```

Each **action** represents a structural change to the table. Common actions include: - `ADD` – add a new column or constraint - `DROP` – remove a column or constraint - `MODIFY` – change the datatype or properties of a column - `RENAME` – rename a table or column

5.4.1 Adding and Removing Columns

5.4.1.1 Adding Columns

New columns can be added to an existing table using the `ADD` clause of the `ALTER TABLE` command.

Syntax:

```
ALTER TABLE tablename
ADD column_definition;
```

Here, `column_definition` follows the same format used when defining columns in a `CREATE TABLE` statement.

Example:

```
ALTER TABLE Customers
ADD PhoneNumber VARCHAR(10);
```

- This command adds a new column `PhoneNumber` to the `Customers` table.

Multiple columns can also be added in a single statement:

```
ALTER TABLE tablename
ADD column1_definition,
ADD column2_definition;
```

5.4.1.2 Removing Columns

Existing columns can be removed from a table using the `DROP COLUMN` command.

- **Important Note:** Removing a column permanently deletes all data stored in that column. This must be done carefully.

Syntax:

```
ALTER TABLE tablename
DROP COLUMN column_name;
```

Example:

```
ALTER TABLE Customers
DROP COLUMN PhoneNumber;
```

- This removes the `PhoneNumber` column from the `Customers` table.

Multiple columns can also be removed in a single statement:

```
ALTER TABLE tablename
DROP COLUMN column1,
DROP COLUMN column2;
```

5.4.2 Adding/Removing Constraints

5.4.2.1 Adding Constraints

Constraints can be added to existing tables using the `ADD` clause.

Syntax:

```
ALTER TABLE tablename
ADD constraint_definition;
```

Here, `constraint_definition` specifies the type of constraint and the column(s) it applies to.

Example (adding a primary key):

```
ALTER TABLE Customers
ADD PRIMARY KEY (CustomerId);
```

This command adds a **primary key constraint** to the `CustomerId` column of the `Customers` table.

5.4.2.2 Removing Constraints

Constraints can also be removed from a table using the `DROP` clause.

Syntax:

```
ALTER TABLE tablename
DROP constraint_type;
```

Example (removing a primary key):

```
ALTER TABLE Customers
DROP PRIMARY KEY;
```

This command removes the **primary key constraint** from the `Customers` table.

5.4.2.2.1 Notes

- A table can have only **one primary key constraint**. Removing it will remove the uniqueness and non-null requirement enforced on the column(s).
- The **foreign key** constraint can be dropped if it was named using the `DROP FOREIGN KEY` clause:

  ```
  ALTER TABLE Orders
  DROP FOREIGN KEY fk_customer;
  ```

- In newer versions of MySQL, other named constraints can be dropped using the `DROP CONSTRAINT` clause:

  ```
  ALTER TABLE Orders
  DROP CONSTRAINT pk_orders;
  ```

5.4.3 Renaming and Modifying Columns

The `ALTER TABLE` clause can also be used to rename/modify columns.

5.4.3.1 Renaming a Column

A column can be renamed using the `RENAME COLUMN` clause.

Syntax:

```
ALTER TABLE tablename
RENAME COLUMN old_column_name TO new_column_name;
```

Example:

```
ALTER TABLE Customers
RENAME COLUMN PhoneNumber TO ContactNumber;
```

5.4.3.2 Modifying a Column

The datatype or constraints of a column can be changed using the `MODIFY` clause.

Syntax:

```
ALTER TABLE tablename
MODIFY column_definition;
```

Here, `column_definition` follows the same format used when defining columns in a `CREATE TABLE` statement.

Example:

```
ALTER TABLE Customers
MODIFY PhoneNumber VARCHAR(15);
```

This changes the datatype of `PhoneNumber` to `VARCHAR(15)`.

5.5 Deleting Tables

A table can be removed from the database using the `DROP TABLE` command. This deletes the **table structure as well as all the data stored in it**.

Syntax:

```
DROP TABLE tablename;
```

Example:

```
DROP TABLE Customers;
```

5.5.1 Important Note

Once a table is dropped, its data **cannot be recovered** unless a backup exists.

6 Data Manipulation

- Once we've created tables, we must add the **data**, which forms the primary part of the database
- We should also be able to update and delete data according to our requirements.
- These operations are handled by the **DML** commands, as discussed earlier.

6.1 Inserting Data

- Insertion of data into tables is done using the `INSERT` command. This command allows you to add rows having values for *all columns*, or having values for only a *few specified columns*.

6.1.1 Inserting Values for All Columns

Syntax:

```
INSERT INTO tablename VALUES tuple1, tuple2 ...
```

- In the above method, values for all columns **must** be specified, and **in the same order** as the columns are in the table.

Each tuple has to be written as:

```
(value1, value2, ...)
```

- String and date values must be enclosed in single `'` or double quotes `"`, while numeric values must be entered as is. `NULL` can be inserted by typing `NULL` without quotes in the tuple.

Example:

```
INSERT INTO Customers VALUES
('C001', 'Alice John', '12 Park Street', '9876543210', 'alice@email.com',
 ↪  '1995-04-12'),
('C002', 'Rahul Sharma', '45 Lake Road', '9123456780', 'rahul@email.com',
 ↪  '1992-11-08');
```

6.1.2 Inserting Values for Selected Columns

- Sometimes it is not necessary to provide values for **all columns**. In such cases, the column names can be specified explicitly.
- If a column is not listed in the column list, it will **automatically** receive `NULL` or its **default value** if set.

- Only column names having a **valid default parameter** can be omitted while inserting records, e.g. if the default value for a column `Email` is `NULL`, but the column has a `NOT NULL` constraint, trying to insert a record without the `Email` column will result in an error.

Syntax:

```
INSERT INTO tablename (column1, column2, ... )
VALUES (value1, value2, ... );
```

In this method:

- Only the specified columns receive values.
- The values must be given **in the same order as the listed columns**, but the order of the listed columns can be different from that of the table.

Example:

```
INSERT INTO Customers (CustomerId, CustomerName, Email, Address)
VALUES ('C003', 'Neha Gupta', 'neha@email.com', '27 Ocean Avenue');
```

In this case:

- Only `CustomerId`, `CustomerName`, `Email`, and `Address` are given values.
- The remaining columns will receive **NULL values or their default values**, if defined.

6.2 Updating Data

- Existing records in a table can be modified using the `UPDATE` command.
- This command changes the values of one or more columns for selected rows.

Syntax:

```
UPDATE tablename
SET column1 = value1,
    column2 = value2,
    ...
[WHERE condition];
```

- The `SET` clause specifies **which columns should be updated** and their **new values**.

- The `WHERE` clause determines **which rows are affected**.

6.2.1 Examples

Single column updation:

```sql
UPDATE Customers
SET PhoneNumber = '9001234567'
WHERE CustomerId = 'C001';
```

This updates the phone number of the customer with ID `C001`.

Multiple column updation:

```sql
UPDATE Customers
SET Address = '78 Green Road',
    PhoneNumber = '9988776655'
WHERE CustomerId = 'C002';
```

Using expressions:

```sql
UPDATE Employees
SET Salary = Salary + 20000
WHERE Department = 'AI';
```

- Expressions can be used to update columns based on their previous value.

6.2.2 Important Note

- If the `WHERE` clause is **omitted**, the update will affect **all rows in the table**.

Example:

```sql
UPDATE Customers
SET Address = 'Unknown';
```

This changes the `Address` for **every record** in the table.

6.3 Deleting Data

- Records in a table can be removed using the `DELETE` command.
- This command deletes **one or more rows** from a table.

Syntax:

```sql
DELETE FROM tablename
[WHERE condition];
```

6.3.1 Examples

6.3.1.1 Single record deletion:

```
DELETE FROM Customers
WHERE CustomerId = 'C003';
```

This deletes the record of the customer whose ID is `C003`.

6.3.1.2 Multiple record deletion:

```
DELETE FROM Customers
WHERE Address = 'Unknown';
```

This deletes all customers whose address is `"Unknown"`.

6.3.2 Important Note

- If the `WHERE` clause is **not specified**, **all rows in the table will be deleted**.

Example:

```
DELETE FROM Customers;
```

- This removes **every record** from the table, but the **table structure remains unchanged**.

Another way of doing the same action is by using the `TRUNCATE` command:

Syntax:

```
TRUNCATE tablename;
```

- Note: Although the `TRUNCATE` command is manipulating the data, it is classified as a **DDL** command, the reason for this is quite complicated — you can read more about it with a quick google.

7 Introduction to Queries

- Data from a database can be accessed using queries. This is done using the `SELECT` command.
- Firstly, we will examine how the `SELECT` command can be used **without referencing a table** to perform arithmetic calculations and evaluate expressions. This feature is often used by developers to **test expressions or verify query logic** before applying them to actual table data.
- In the next chapter, we will deal with simple queries such as accessing all records of a table, as well as complex ones — querying data from multiple tables, which can be handled using different clauses of the `SELECT` command.
- MySQL also provides a **dummy table** called `DUAL`. This exists mainly for compatibility with the SQL standard and with other database systems that require a query to select data **from a table**.
- Any query that can be executed without referencing a table can also be written by selecting from the `DUAL` table using the `FROM` clause.

7.1 Arithmetic and Logical Operations

- The `SELECT` command can evaluate **arithmetic and logical expressions** directly. The result is given in the form of a table having one column and one row.
- Logical expressions return `1` for true and `0` for false.

Basic operators supported by SQL include:

Operator	Meaning
`+`	Addition
`-`	Subtraction
`*`	Multiplication
`/`	Division
`%`	Modulus (remainder)
`=`	Is equal to
`!=` or `<>`	Is not equal to
`>`	Greater than
`<`	Lesser than
`>=`	Greater than or equal to
`<=`	Lesser than or equal to
`AND`	True if both conditions are true
`OR`	True if at least one condition is true
`NOT`	Reverses the result of a condition

Examples:

```
mysql> SELECT 6 * 7;
+--------+
| 6 * 7  |
+--------+
| 42     |
+--------+
```

```
mysql> SELECT 14!=25;
+--------+
| 14!=25 |
+--------+
|      1 |
+--------+
```

- Multiple operators can be used in an expression, which will be evaluated in the standard mathematical order of precedence.

  Parentheses can be used to control the evaluation order — e.g. `SELECT 5 + 3 * 2;` results in `11`, and `SELECT (5 + 3) * 2;` results in `16`

To change the column name (which is by default just the expression passed to the `SELECT` command), we can use an **alias**. This will be discussed further later when dealing with tables.

Example:

```
mysql> SELECT 5 + 3 AS Result;
+--------+
| Result |
+--------+
| 8      |
+--------+
```

Multiple expressions can be evaluated at once by separating them with **commas**. This comma-separated syntax is the same pattern used to select **multiple columns** from a table, which we will explore further sections.

Example:

```
mysql> SELECT 5 + 3, 10 - 4, 6 * 7;
+-------+--------+--------+
| 5 + 3 | 10 - 4 | 6 * 7  |
+-------+--------+--------+
| 8     | 6      | 42     |
+-------+--------+--------+
```

7.2 Working with Strings

- Select can also be used to return string values.

Example:

```
mysql> SELECT 'Hello World';
+-------------+
| Hello World |
+-------------+
| Hello World |
+-------------+
```

- Generally, this is not very useful, however, we can use built-in functions, that work on text, and other data types to manipulate them as required.

7.3 MySQL's Built-In Functions

- These are **special predefined command sets**, that perform a **specific operation** on a value or a column, and **return a result for each value**.
- Using `SELECT` without a table allows the developer to **test the usage of built-in functions**, before applying it on a table.
- Some commonly used built-in functions for different datatypes are listed below.

7.3.1 Numeric Functions

Function	Description	Example	Result
`ABS(n)`	Returns the absolute value of n	`ABS(-15)`	15
`ROUND(n, d)`	Rounds n to d decimal places	`ROUND(3.14159, 2)`	3.14
`CEIL(n)`	Returns the smallest integer greater than or equal to n	`CEIL(4.2)`	5
`FLOOR(n)`	Returns the largest integer less than or equal to n	`FLOOR(4.9)`	4
`TRUNCATE(n, d)`	Truncates n to d decimal places without rounding	`TRUNCATE(3.14159, 2)`	3.14
`MOD(n, m)`	Returns the remainder when n is divided by m	`MOD(17, 5)`	2
`POWER(n, e)`	Returns n raised to the power e	`POWER(2, 8)`	256
`SQRT(n)`	Returns the square root of n	`SQRT(144)`	12

7.3.2 Date/Time Functions

Function	Description	Example	Result
`NOW()`	Returns the current date and time	`NOW()`	2026-03-24 10:45:00

Function	Description	Example	Result
`CURDATE()`	Returns the current date	`CURDATE()`	2026-03-24
`CURTIME()`	Returns the current time	`CURTIME()`	10:45:00
`YEAR(date)`	Extracts the year from a date	`YEAR('2026-03-24')`	2026
`MONTH(date)`	Extracts the month from a date	`MONTH('2026-03-24')`	3
`DAY(date)`	Extracts the day from a date	`DAY('2026-03-24')`	24
`MONTHNAME(date)`	Returns the full name of the month	`MONTHNAME('2026-03-24')`	March
`DAYNAME(date)`	Returns the name of the weekday	`DAYNAME('2026-03-24')`	Tuesday
`DATEDIFF(d1, d2)`	Returns the number of days between two dates	`DATEDIFF('2026-12-31', '2026-03-24')`	282

7.3.3 String Functions

Function	Description	Example	Result
`UPPER(str)`	Converts string to uppercase	`UPPER('hello')`	HELLO
`LOWER(str)`	Converts string to lowercase	`LOWER('HELLO')`	hello
`LENGTH(str)`	Returns the length of the string in bytes	`LENGTH('hello')`	5
`CONCAT(s1, s2, ...)`	Joins two or more strings together	`CONCAT('Hello', ' ', 'World')`	Hello World
`SUBSTRING(str, pos, len)`	Extracts a substring starting at `pos`, of length `len`	`SUBSTRING('Hello World', 7, 5)`	World
`TRIM(str)`	Removes leading and trailing spaces	`TRIM(' hello ')`	hello
`LTRIM(str)`	Removes leading (left) spaces	`LTRIM(' hello')`	hello
`RTRIM(str)`	Removes trailing (right) spaces	`RTRIM('hello ')`	hello
`REPLACE(str, from, to)`	Replaces all occurrences of a substring with another	`REPLACE('Hello World', 'World', 'SQL')`	Hello SQL
`REVERSE(str)`	Reverses the string	`REVERSE('hello')`	olleh
`INSTR(str, substr)`	Returns the position of the first occurrence of `substr` in `str`	`INSTR('Hello World', 'World')`	7
`LEFT(str, n)`	Returns the leftmost n characters	`LEFT('Hello World', 5)`	Hello
`RIGHT(str, n)`	Returns the rightmost n characters	`RIGHT('Hello World', 5)`	World

Note: For functions such as `SUBSTRING`, indexing starts at 1, which is different from most programming languages which start indexing at 0.

7.3.4 Miscellaneous

Function	Description	Example	Result
`IFNULL(value, fallback)`	If the `value` field is null, it returns the `fallback`	`IFNULL(NULL,'nothing')`	nothing

7.3.5 Using Built-In functions without a table

- Here are some examples where built-in functions are used without referencing a table.

```
mysql> SELECT ROUND(3.14159, 2);
+-------------------+
| ROUND(3.14159, 2) |
+-------------------+
|              3.14 |
+-------------------+
```

```
mysql> SELECT UPPER('hello'), ROUND(3.14159, 2) AS Rounded, DAYNAME(CURDATE())
 ↪  AS Today;
+----------------+---------+-----------+
| UPPER('hello') | Rounded | Today     |
+----------------+---------+-----------+
| HELLO          |    3.14 | Tuesday   |
+----------------+---------+-----------+
```

- As seen in the examples above, multiple functions can be combined in a single `SELECT` statement by separating them with commas.
- Aliases can be used with functions just as with any other expression, using the `AS` keyword, to make the output column names more readable.

7.3.6 Importance

- These functions become especially powerful **when applied to columns of an actual table** — for instance, using `UPPER()` on a name column, or `DATEDIFF()` to calculate a student's age from a date of birth column. We will see such examples in the upcoming sections.

8 Querying Individual Tables

- The SELECT command is used to retrieve data from one or more tables in a database. Its full syntax supports several optional clauses, each serving a specific purpose. Not all clauses are required — a simple query may use only a few of them, while more complex queries may chain several together.

The general structure of a SELECT statement is as follows:

```
SELECT   column1, column2, ...     <- Columns to retrieve. Use * for all.
FROM     tablename                 <- Table to retrieve data from.
WHERE    condition                 ┐
GROUP BY column                    │
HAVING   condition                 ├ All optional clauses
ORDER BY column [ASC|DESC]         │
LIMIT    n;                        ┘
```

Only the SELECT and FROM[1] are mandatory — all other clauses are optional and can be included as needed. The clauses must always be written in the order shown above. In the sections that follow, each clause will be explored in detail.

8.1 Selecting Unique Values

- To select unique/distinct values of a column, or select distinct pairs of values across multiple columns, the SELECT DISTINCT command can be used.

Syntax:

```
SELECT DISTINCT column1, column2 ... FROM tablename;
```

- Note: If multiple columns are specified, the statement evaluates the **unique combination** of all specified columns.

8.2 Specifying Columns

After the SELECT keyword, specify the columns you want to retrieve as a comma-separated list. For example, SELECT name, age, city retrieves only those three columns. To retrieve all columns in the table, use an asterisk (*) in place of the column list — SELECT *. While convenient, using SELECT * is generally discouraged in practice as it retrieves unnecessary data; it is better to explicitly name only the columns you need.

Beyond plain column names, SELECT can also include:

[1] The FROM clause is only mandatory when selecting data from a table.

- **Built-in functions applied to columns** —

  For example, `SELECT UPPER(name), LENGTH(email)` applies a function to each value in that column as it is retrieved.

- **Arithmetic expressions on columns** —

  For example, `SELECT price * quantity` computes a value from two columns for each row.

- **Constants and expressions** —

  For example, `SELECT name, 'Student' AS type` includes a fixed string value alongside a column for every row returned.

These can all be freely mixed in a single `SELECT` statement, separated by commas, and aliases can be used with any of them to give the output column a meaningful name.

> The `IFNULL` function, discussed earlier, can be used to display `NULL` values in a more user friendly way: `SELECT Name, IFNULL(PhoneNumber, 'Unavailable') FROM Contacts` This would display the phone numbers of the contacts, and display 'Unavailable' for those contacts which do not have a phone number.

8.2.1 Aliases

- An alias gives a column or expression in the result a custom name, using the `AS` keyword. For example, `SELECT UPPER(name) AS Name, price * quantity AS Total` labels the two output columns as `Name` and `Total` rather than the default, which would just be the expression itself.

  The `AS` keyword is optional — `SELECT UPPER(name) Name` works the same way — however, including it makes the query easier to read.

- Note that aliases cannot be used in the `WHERE` clause of the same query, as `WHERE` is evaluated before the `SELECT` columns are processed. However, MySQL does allow aliases to be referenced in `ORDER BY`, `GROUP BY`, and `HAVING` — though this is a MySQL-specific behaviour and is not part of the standard SQL specification.

8.3 The FROM Clause

- The `FROM` clause specifies the table name from which the data is to be queried. In later sections we will see how `FROM` can also be used to query multiple tables at once using joins.

8.4 The WHERE Clause

- The `WHERE` clause filters the rows returned by a query, retrieving only those that satisfy a given condition.
- The condition is an expression that evaluates to true or false for each row — only rows for which it evaluates to true are included in the result.
- Conditions are built using relational operators (`=`, `!=`, `>`, `<`, `>=`, `<=`) to compare column values against other values or expressions, and multiple conditions can be combined using the logical operators `AND`, `OR`, and `NOT`.

Example:

```
mysql> SELECT name, age FROM students WHERE age > 18;
+---------+-----+
| name    | age |
+---------+-----+
| Alice   |  19 |
| Bob     |  21 |
| Charlie |  20 |
+---------+-----+
```

- The above query selects the name and age of students whose age is greater than 18 from the `students` table.

Multiple conditions can also be combined using logical operators `AND`, `OR` and `NOT`

Example:

```
mysql> SELECT name, age, city FROM students WHERE age > 18 AND city =
 ↪  'London';
+---------+-----+--------+
| name    | age | city   |
+---------+-----+--------+
| Alice   |  19 | London |
| Charlie |  20 | London |
+---------+-----+--------+
2 rows in set (0.00 sec)
```

These are special conditions used in the `WHERE` clause for cases that cannot be handled with regular relational operators:

Condition	Meaning	Example
`IS NULL`	True if the column value is null	`WHERE email IS NULL`
`IS NOT NULL`	True if the column value is not null	`WHERE email IS NOT NULL`
`LIKE pattern`	True if the value matches a pattern	`WHERE name LIKE 'A_%'`
`IN (v1, v2, ...)`	True if the value matches any value in the list	`WHERE city IN ('London', 'Paris')`
`BETWEEN v1 AND v2`	True if the value is within the range (inclusive)	`WHERE age BETWEEN 18 AND 25`

8.4.1 The LIKE operator

- The `LIKE` operator is used in the `WHERE` clause to filter rows based on a pattern match against a string column, rather than an exact value. It uses two wildcard characters:

Wildcard	Meaning
%	Matches any sequence of zero or more characters
_	Matches exactly one character

Examples:

```
mysql> SELECT name FROM students WHERE name LIKE 'A%';
+-------+
| name  |
+-------+
| Alice |
| Aaron |
+-------+
```

```
mysql> SELECT name FROM students WHERE name LIKE '_ob';
+------+
| name |
+------+
| Bob  |
| Rob  |
+------+
```

- The first query returns all students whose name begins with `A`. The second returns all students whose name is exactly three characters long and ends with `ob`.
- Patterns can also use both wildcards together — for example, `LIKE '_a%'` would match any name where the second character is `a`.

8.5 Aggregate Functions and the GROUP BY Clause

- Aggregate functions are functions that accept values of multiple rows and return a single result. Using the `GROUP BY` clause, the rows can be divided into groups and passed to the aggregate function, allowing the result to be displayed for each group of rows.

Syntax:

```
SELECT column1, AggregateFunction(column2) FROM tablename GROUP BY column1;
```

- The columns that do not have aggregate functions applied to them must be grouped using the `GROUP BY` clause.
- If only the column with an aggregate function applied is selected, only one result is obtained.

Some common aggregate functions of MySQL are:

Function	Syntax	Description	Example
`AVG()`	`AVG(column_name)`	Calculates the average value of a numeric column.	`SELECT AVG(price) FROM products;`
`COUNT()`	`COUNT(column_name)` or `COUNT(*)`	Returns the number of rows that match a specified criterion.	`SELECT COUNT(orderid) FROM orders;`
`MAX()`	`MAX(column_name)`	Returns the largest value in the selected column.	`SELECT MAX(amount) FROM orders;`
`MIN()`	`MIN(column_name)`	Returns the smallest value in the selected column.	`SELECT MIN(amount) FROM orders;`
`SUM()`	`SUM(column_name)`	Returns the total sum of a numeric column.	`SELECT SUM(salary) FROM employees;`

8.5.1 Notes

- Most aggregate functions ignore `NULL` values when performing calculations. `COUNT()` only counts rows with `NULL` values when `COUNT(*)` is used.
- Distinct Calculation: You can use the `DISTINCT` keyword inside these functions (e.g., `COUNT(DISTINCT column)`) to perform calculations on unique values only.
- Numeric requirement: Functions like `AVG()` and `SUM()` require numeric data, while `MIN()` and `MAX()` can also work with strings and dates.

8.5.2 Example

Sample Table: Orders

OrderId	CustomerName	Category	Amount
1	Alice	Electronics	1200
2	Bob	Clothing	50
3	Alice	Clothing	80
4	Charlie	Electronics	400
5	Bob	Clothing	30

To find out how many orders each customer has made, we can perform the following query:

```
mysql> SELECT CustomerName, COUNT(OrderId) AS TotalOrders FROM Orders GROUP BY
 ↪  CustomerName;
+--------------+-------------+
| CustomerName | TotalOrders |
+--------------+-------------+
| Alice        |           2 |
| Bob          |           2 |
| Charlie      |           1 |
+--------------+-------------+
```

8.6 The HAVING Clause

- The `HAVING` clause works similarly to the `WHERE` clause, but with one key difference — The `WHERE` clause acts on the **data stored in the table**, while the HAVING clause acts on the **data retrieved after grouping**.
- Thus, it can be used to further filter the grouped results obtained using the `GROUP BY` clause.

8.6.1 Example

Using the same sample table **Orders** from above, we can perform the following query:

```
mysql> SELECT Category, SUM(Amount) AS TotalAmt FROM Orders GROUP BY Category
 ↪  HAVING TotalAmt < 1000;
+----------+----------+
| Category | TotalAmt |
+----------+----------+
| Clothing |      160 |
+----------+----------+
```

Here, only the category having total amount less than 1000 has been displayed.

8.6.2 Common Mistakes

- A common mistake is to use the `WHERE` clause instead of the `HAVING` clause. Remember, the `WHERE` clause cannot act on data after it has been grouped, but can only be used to filter data before grouping.

8.6.3 Using both WHERE and HAVING

- For cases where a developer wants to first filter by some condition, then group the data, then show results where the aggregate function value matches some condition, both `WHERE` and `HAVING` clauses need to be used. For example, if a developer wants to find the products having more than 1000 sales in a particular city, the query would be something like:

```
SELECT ProductName, COUNT(ProductId) as SoldCount
FROM SoldProducts
WHERE City = 'Bengaluru'
HAVING SoldCount > 1000;
```

8.7 The ORDER BY Clause

- It is used to order data selected in a table in either ascending `ASC` (default) or descending `DESC` order.
- By default, the `ORDER BY` clause is case insensitive, thus if it is used on a text column, ‘apple’ and ‘Apple’ will be displayed in the order that they have been inserted into the column.
- The `DESC` keyword can be written after the expression for descending order.

8.7.1 Examples

```
SELECT Name FROM Employees ORDER BY LENGTH(Name);
```

```
SELECT Name, Price FROM Products ORDER BY Price DESC;
```

This clause can be used on the result of a GROUP BY clause, for example (Sample table **Orders**):

```
mysql> SELECT CustomerName, SUM(Amount) AS TotalAmt FROM Orders GROUP BY
 ↪  CustomerName ORDER BY TotalAmt DESC;
+---------------+----------+
| CustomerName  | TotalAmt |
+---------------+----------+
| Alice         |     1280 |
| Charlie       |      400 |
| Bob           |       80 |
+---------------+----------+
3 rows in set (0.00 sec)
```

8.8 The LIMIT Clause

- It is used to limit the number of results of the query.

8.8.1 Examples

To get the top 10 most populated cities:

```
SELECT State, City, Population FROM Cities ORDER BY Population DESC LIMIT 10;
```

To get the least spender from the **Orders** sample table:

```
mysql> SELECT CustomerName, SUM(Amount) AS TotalAmt FROM Orders GROUP BY
 ↪  CustomerName ORDER BY TotalAmt LIMIT 1;
+--------------+----------+
| CustomerName | TotalAmt |
+--------------+----------+
| Bob          |       80 |
+--------------+----------+
```

8.9 Order of Execution of the SELECT command

The logical order of execution is different from how you write the query. MySQL processes them in this specific sequence:

1. **`FROM`** (and **`JOIN`**): First, the database identifies the table(s) to pull data from.
2. **`WHERE`**: Filters the raw rows ***before*** any grouping or calculations happen.
3. **`GROUP BY`**: Collapses the remaining rows into buckets based on the specified columns.
4. **`HAVING`**: Filters the ***groups*** created in the previous step (can use aggregate results).
5. **`SELECT`**: Determines which columns to show and calculates any expressions/aliases.
6. **`ORDER BY`**: Sorts the final result set for display.
7. **`LIMIT`**: Finally, chops off the results to only show the requested number of rows.

Here, the difference between the `WHERE` and `HAVING` clause is more clear, and it can be understood why they cannot be used interchangeably.

9 Querying Multiple Tables — Joins

9.1 Introduction to Joins

In relational databases, data is often stored across multiple tables to reduce redundancy and maintain data integrity. **Joins** are SQL operations that allow us to combine rows from two or more tables based on related columns between them.

All joins in SQL are fundamentally derived from the **Cartesian product** (cross join) of two tables. Other join types—such as equi-join and natural join—are simply filtered or refined versions of this cross product.

9.2 Cross Join (Cartesian Product)

- A cross join is the cartesian product of the two tables, i.e. it is a table consisting of all possible combinations of rows of the two tables.
- If the first table has n rows and a columns, and the second has m rows and b columns, the cross join of the two will have $n \times m$ rows and $a + b$ columns. This will be clearer in the following example.

9.2.1 Syntax

```
SELECT * FROM table1 CROSS JOIN table2;
```

Or alternatively:

```
SELECT * FROM table1, table2;
```

9.2.2 Example

Consider two tables:

A) Students:

StudentID	StudentName
1	Alice
2	Bob
3	Charlie

B) Courses:

CourseID	CourseName
101	Math
102	Science

```
mysql> SELECT * FROM Students CROSS JOIN Courses;
+-----------+-------------+----------+------------+
| StudentID | StudentName | CourseID | CourseName |
+-----------+-------------+----------+------------+
|         1 | Alice       |      101 | Math       |
|         1 | Alice       |      102 | Science    |
|         2 | Bob         |      101 | Math       |
|         2 | Bob         |      102 | Science    |
|         3 | Charlie     |      101 | Math       |
|         3 | Charlie     |      102 | Science    |
+-----------+-------------+----------+------------+
```

The result has $3 \times 2 = 6$ rows and $2 + 2 = 4$ columns.

- Cross joins are rarely used practically, as they result in large result sets. (e.g. if two tables have 1000 rows each, their cross join would have 1,000,000 rows).
- They can, however be used for some specific use cases:
  - Generating all possible combinations
  - Statistical/Mathematical calculations
  - Creating large amounts of test data

9.3 Equi-Join — Joining based on an equality condition

- The equi-join is a fundamental operation in database management. It enables us to **combine data from multiple tables** based on **matching values between their columns**, which is essential for querying relational databases.
- Equi-join (and other joins) are **subsets of the cross join**, as they are just rows of the cross join result which satisfy the equality condition defined by the developer.

9.3.1 Classification

Equi-joins can be classified into two main types:

1. Inner Join (`INNER JOIN` or `JOIN`)

- Returns only the rows where there is a match in **both tables**.
- If a row in one table has no matching value in the other table, it is excluded from the result.
- This is the most commonly used type of join.

2. Outer Join

- Returns matching rows **plus** unmatched rows from one or both tables.

- Unmatched rows are filled with NULL values for columns from the table that has no match.
- Outer joins are further divided into:
  - `LEFT JOIN` - Returns all rows from the left table and matching rows from the right table..
  - `RIGHT JOIN` - Returns all rows from the right table and matching rows from the left table.
  - **FULL OUTER JOIN** - Returns all rows from both tables (matching and non-matching).

Note: MySQL does **not support FULL OUTER JOIN** natively. However, it can be simulated using a combination of LEFT JOIN and RIGHT JOIN with a UNION operator.

9.3.2 Syntax

```
SELECT * FROM table1 INNER JOIN table2 ON table1.common_col =
↪  table2.common_col;
```

Here, `INNER JOIN` can be replaced by the type of join that is desired as above. Note: `INNER JOIN` and `JOIN` refer to the same functionality, and can be used interchangeably.

Or alternatively (for inner join):

```
SELECT * FROM table1, table2 WHERE table1.common_col = table2.common_col;
```

9.3.3 Example

- If we stored a *package ID* for each customer who has ordered a package in a **Customers** table, and we want to retrieve the ***list of customer names*** along with the ***name of the package*** they have ordered, we must query both the **Customers** table for the customer names, and the **Packages** table for the package names. Here, we must only display records where the package ID of the package corresponds to the package that the customer has ordered.

Consider the two tables:

A) Customers:

CustomerID	CustomerName	PackageID
1	Alice	101
2	Bob	102
3	Charlie	101
4	Diana	NULL

B) Packages:

PackageID	PackageName
101	Basic Plan

PackageID	PackageName
102	Premium Plan
103	Enterprise Plan

The query can be written as:

```
SELECT Customers.CustomerName, Packages.PackageName
FROM Customers, Packages
WHERE Customers.PackageID = Packages.PackageID;
```

Or using explicit `INNER JOIN` syntax:

```
SELECT Customers.CustomerName, Packages.PackageName
FROM Customers
INNER JOIN Packages ON Customers.PackageID = Packages.PackageID;
```

The result would thus be:

```
+--------------+-----------------+
| CustomerName | PackageName     |
+--------------+-----------------+
| Alice        | Basic Plan      |
| Bob          | Premium Plan    |
| Charlie      | Basic Plan      |
+--------------+-----------------+
```

Note: Differences that would've been observed when using `LEFT JOIN` - Since the customer "Diana" does not have a purchased package, the result does not show her package name. - However, if `LEFT JOIN` was used instead of `INNER JOIN`, the result would include "Diana", with her package name being `NULL`.

9.3.4 Table Aliases

- When using equi-joins, while mentioning column names, we must also mention the table the column belongs to, as there may be common column names in the tables (the table name and column name are to be separated by a `.` as seen in the above example).
- When many columns are to be selected, the command can become unnecessarily long. To prevent this, we can use table aliases, which have a similar syntax to column aliases we saw earlier.

Syntax:

```
SELECT t1.col1, t2.col2 FROM table1 AS t1 INNER JOIN table2 AS t2 ON
↪  t1.common_col = t2.common_col;
```

Note: The `AS` keyword is optional here as well.

9.4 Natural Join

A **natural join** is a specialized form of equi-join that automatically joins two tables based on all columns that share the same name in both tables. Unlike a standard equi-join where the join condition must be explicitly specified, a natural join performs the comparison implicitly.

- The database identifies columns with identical names in both tables, and uses the equality operator on the values to join the tables.
- The identically named column only appears once in the result.
- No explicit join condition (ON or WHERE clause) is required

9.4.1 Syntax

```
SELECT * FROM table1 NATURAL JOIN table2;
```

9.4.2 Example

If both the **Students** table and the **Enrollments** table contain a column named “StudentID”, MySQL will automatically join the tables on this column and display StudentID only once in the result.

```
SELECT * FROM Students NATURAL JOIN Enrollments;
```

9.5 Note

- Since the `JOIN` is evaluated on the raw data on the table, the columns having matching names need not be selected while performing a natural join. i.e., if we did not want to display the StudentID in the above table, only the columns that we want to display can be mentioned, and the join would work as intended.
- Natural joins can produce unexpected results if tables contain unintended common column names. It is generally recommended to use explicit join conditions for better control and clarity in production queries.

---

10 Afterword

Most of this book, was written by me just a few days before our board exams, as a way to revise the SQL syllabus. I was unable to include a section on Python Connectivity due to time constraints. This may be added in the future, if I find time for it.

10.1 Some More Information

For accurate information about MySQL's functions, datatypes, etc., visit the MySQL Documentation. Some features however, may not be a part of the syllabus.

Another prominent RDBMS software is PostgreSQL, which is used extensively in common applications such as Instagram, Reddit, Spotify, etc. Postgres too is a Relational Database, and uses SQL to query and modify data.

You can learn more about it online, and maybe use it for personal projects. (Do confirm with your teacher whether it can be used for the CBSE Board Project if you want to do that)

Google's Firebase platform uses a 'NoSQL' database, which has a flexible structure, and can adapt to the application's requirements. Many other database solutions also use 'NoSQL', for different reasons such as performance or flexibility. Some popular ones are MongoDB and Redis

10.2 Errors in the Handbook

If you encounter any errors in the book itself or other issues while accessing the book, do reach out to me at the email address given in the **Preface** section, or create an Issue on GitHub.